🚀 COMPLETE BACKEND DOCUMENTATION — REALTIME LUDO GAME

Project

Realtime Ludo Game Backend

📌 BACKEND TECH STACK

Core Technologies

- Node.js
- Express.js
- MongoDB
- Mongoose
- Socket.io
- Firebase Admin SDK
- JWT Authentication
- Redis (future scaling)

📌 REQUIRED NPM PACKAGES

Core Dependencies

```
npm i express mongoose cors dotenv jsonwebtoken bcrypt firebase-admin
socket.io uuid
```

Security Packages

```
npm i helmet express-rate-limit xss-clean express-mongo-sanitize
```

Validation Packages

```
npm i joi express-validator
```

Logging Packages

```
npm i morgan winston
```

Utility Packages

```
npm i dayjs lodash
```

Development Packages

```
npm i -D nodemon
```

Future Scaling Packages

```
npm i redis @socket.io/redis-adapter ioredis
```

📌 BACKEND FOLDER STRUCTURE

```
backend/
│
├── src/
│   ├── config/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   ├── routes/
```

```
│   ├── sockets/
│   ├── middlewares/
│   ├── validators/
│   ├── constants/
│   ├── utils/
│   ├── jobs/
│   ├── cron/
│   ├── logs/
│   ├── docs/
│   ├── app.js
│   └── server.js
│
├── .env
├── package.json
└── README.md
```

📌 ENV VARIABLES

```
PORT=5000
NODE_ENV=development

MONGO_URI=YOUR_MONGO_URI

JWT_SECRET=YOUR_SECRET
JWT_EXPIRE=7d

FIREBASE_PROJECT_ID=PROJECT_ID
FIREBASE_PRIVATE_KEY=PRIVATE_KEY
FIREBASE_CLIENT_EMAIL=CLIENT_EMAIL

SOCKET_PORT=5001

REDIS_URL=YOUR_REDIS_URL

CLIENT_URL=http://localhost:3000
ADMIN_URL=http://localhost:3001
```

📌 BASE API URL

```
/api/v1
```

📌 STANDARD API RESPONSE FORMAT

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

Error Response

```
{
  "success": false,
  "message": "Something went wrong",
  "error": {}
}
```

📌 AUTH MODULE APIs

1. Verify Firebase Login

Endpoint

```
POST /auth/verify
```

Purpose

- Verify firebase token
- Create user if not exists
- Generate JWT token

Request

```
{
  "firebaseToken": "firebase_token"
}
```

Response

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "jwt_token",
    "user": {}
  }
}
```

---

2. Refresh JWT Token

Endpoint

```
POST /auth/refresh-token
```

---

3. Logout User

Endpoint

```
POST /auth/logout
```

---

4. Get User Profile

Endpoint

```
GET /auth/profile
```

5. Update User Profile

Endpoint

```
PUT /auth/profile
```

---

6. Delete Account

Endpoint

```
DELETE /auth/delete-account
```

📌 WALLET MODULE APIs

7. Get Wallet Balance

Endpoint

```
GET /wallet
```

---

8. Get Wallet Transaction History

Endpoint

```
GET /wallet/history
```

---

9. Add Coins (Admin)

Endpoint

```
POST /wallet/add
```

10. Deduct Coins (Admin)

Endpoint

```
POST /wallet/deduct
```

---

11. Freeze Wallet

Endpoint

```
POST /wallet/freeze
```

---

12. Unfreeze Wallet

Endpoint

```
POST /wallet/unfreeze
```

📌 GAME MODULE APIs

13. Create Practice Game

Endpoint

```
POST /game/practice/create
```

Working

```
Create room
→ Add player
→ Add bot
```

```
→ Initialize board
→ Return game ID
```

---

14. Create Cash Game

Endpoint

```
POST /game/cash/create
```

Request

```
{
  "betAmount": 100
}
```

Working

```
Validate wallet
→ Deduct coins
→ Join matchmaking queue
→ Find opponent
→ Create game room
```

---

15. Join Existing Game

Endpoint

```
POST /game/join
```

---

16. Get Game Details

Endpoint

```
GET /game/:gameId
```

---

17. Get Active Game

Endpoint

```
GET /game/active
```

---

18. Roll Dice

Endpoint

```
POST /game/roll-dice
```

Working

```
Verify turn
→ Generate random dice
→ Save move
→ Emit socket event
```

---

19. Move Token

Endpoint

```
POST /game/move-token
```

Request

```
{
  "gameId": "GAME123",
  "tokenId": "red_1",
  "steps": 6
}
```

Working

```
Validate move
→ Check safe zone
→ Check kill
→ Update board state
→ Save move history
→ Emit realtime update
```

---

20. Skip Turn

Endpoint

```
POST /game/skip-turn
```

---

21. Surrender Match

Endpoint

```
POST /game/surrender
```

---

22. End Game

Endpoint

```
POST /game/end
```

Working

```
Mark winner
→ Update wallet
→ Update stats
→ Save history
→ Emit game end event
```

---

23. Reconnect Game

Endpoint

```
POST /game/reconnect
```

---

24. Restore Game State

Endpoint

```
GET /game/restore/:gameId
```

📌 MATCHMAKING MODULE APIs

25. Join Matchmaking Queue

Endpoint

```
POST /matchmaking/join
```

---

26. Leave Matchmaking Queue

Endpoint

```
POST /matchmaking/leave
```

---

27. Get Queue Status

Endpoint

```
GET /matchmaking/status
```

📌 BOT MODULE APIs

28. Get Bot Move

Endpoint

```
POST /bot/move
```

Working

```
Analyze board
→ Find best move
→ Move token
→ Emit move event
```

---

29. Set Bot Difficulty

Endpoint

```
POST /bot/difficulty
```

📌 CHAT MODULE APIs

30. Send Quick Chat

Endpoint

```
POST /chat/send
```

---

31. Get Chat History

Endpoint

```
GET /chat/:gameId
```

📌 STATS MODULE APIs

32. Get Player Stats

Endpoint

```
GET /stats
```

---

33. Get Match History

Endpoint

```
GET /stats/match-history
```

---

34. Get Win Rate

Endpoint

```
GET /stats/win-rate
```

---

35. Get Leaderboard

Endpoint

```
GET /stats/leaderboard
```

📌 NOTIFICATION MODULE APIs

36. Register Device Token

Endpoint

```
POST /notification/register-device
```

---

37. Send Push Notification

Endpoint

```
POST /notification/send
```

📌 REPORT MODULE APIs

38. Report User

Endpoint

```
POST /report/user
```

---

39. Report Match

Endpoint

```
POST /report/match
```

📌 ADMIN MODULE APIs

40. Admin Login

Endpoint

```
POST /admin/login
```

---

41. Get Dashboard Analytics

Endpoint

```
GET /admin/dashboard
```

---

42. Get All Users

Endpoint

```
GET /admin/users
```

---

43. Get Single User

Endpoint

```
GET /admin/user/:id
```

---

44. Ban User

Endpoint

```
POST /admin/ban-user
```

---

45. Suspend User

Endpoint

```
POST /admin/suspend-user
```

---

46. Get All Games

Endpoint

```
GET /admin/games
```

---

47. Get Live Matches

Endpoint

```
GET /admin/live-games
```

---

48. Force End Match

Endpoint

```
POST /admin/force-end-game
```

---

49. Wallet Adjustment

Endpoint

```
POST /admin/wallet-adjustment
```

---

50. Get Revenue Analytics

Endpoint

```
GET /admin/revenue
```

📌 SOCKET EVENTS DOCUMENTATION

Client → Server Events

```
connection
join_game
leave_game
roll_dice
move_token
```

```
chat_message
reconnect_game
matchmaking_join
matchmaking_leave
```

Server → Client Events

```
game_joined
dice_rolled
token_moved
turn_changed
chat_received
game_ended
player_disconnected
player_reconnected
match_found
wallet_updated
```

📌 DATABASE MODELS REQUIRED

User Model

```
phone
firebaseUid
coins
wins
losses
totalGames
isBanned
createdAt
```

Game Model

```
gameId
players
status
winner
betAmount
boardState
```

```
moves
createdAt
```

Wallet Transaction Model

```
userId
amount
type
reason
createdAt
```

Match History Model

```
winner
loser
gameType
betAmount
duration
createdAt
```

Notification Model

```
userId
title
body
isRead
createdAt
```

Report Model

```
reportedBy
reportedUser
reason
status
createdAt
```

📌 BACKEND MIDDLEWARES REQUIRED

1. Auth Middleware

Purpose:

- Verify JWT
- Protect APIs

---

2. Admin Middleware

Purpose:

- Verify admin role

---

3. Error Middleware

Purpose:

- Handle centralized errors

---

4. Validation Middleware

Purpose:

- Validate request body

---

5. Rate Limit Middleware

Purpose:

- Prevent spam

📌 BACKEND SERVICES REQUIRED

Auth Service

Responsibilities:

- Firebase verification
- JWT generation
- User creation

Wallet Service

Responsibilities:

- Add coins
- Deduct coins
- Validate balance
- Transaction locking

Game Service

Responsibilities:

- Board generation
- Move validation
- Winner detection
- Turn management

Bot Service

Responsibilities:

- Generate bot moves
- Difficulty handling

Matchmaking Service

Responsibilities:

- Queue players

- Match users
- Add bot fallback

Notification Service

Responsibilities:

- Send FCM notifications

📌 BACKEND SECURITY CHECKLIST

Required Security Features

- JWT authentication
- Helmet protection
- Rate limiting
- Mongo sanitize
- XSS protection
- Request validation
- Wallet transaction locking
- Admin role verification

📌 GAME VALIDATION RULES

Backend MUST validate:

- Correct turn
- Valid token
- Safe zones
- Dice values
- Winner logic
- Home entry logic
- Kill logic

Frontend should NEVER validate gameplay.

📌 CRON JOBS REQUIRED

Daily Reward Cron

```
Run every 24 hours
```

Cleanup Cron

```
Remove inactive games
```

Analytics Cron

```
Generate daily reports
```

📌 LOGGING SYSTEM

Log Types

- API logs
- Error logs
- Wallet logs
- Match logs
- Admin activity logs

Recommended Tools

```
Winston
Morgan
```

📌 REDIS USAGE (FUTURE)

Redis Stores:

- Active games
- Matchmaking queue
- Socket sessions
- Online users
- Live board states

📌 PERFORMANCE OPTIMIZATION

Backend Optimizations

- MongoDB indexing
- Lean queries
- Redis caching
- Socket room optimization
- Pagination everywhere

📌 TESTING REQUIREMENTS

API Testing

- Auth APIs
- Wallet APIs
- Game APIs
- Admin APIs

Socket Testing

- Realtime sync
- Reconnect flow
- Duplicate event prevention

Wallet Testing

- Double spend prevention
- Simultaneous transaction handling

📌 DEPLOYMENT REQUIREMENTS

Production Backend Hosting

Recommended:

- AWS EC2
- Render
- Railway

Process Manager

Use:

```
pm2 start server.js
```

Reverse Proxy

Use:

```
NGINX
```

SSL

Use:

```
HTTPS with SSL certificate
```

📌 FINAL BACKEND DEVELOPMENT RULES

Important Rules

- No hardcoding
- Use environment variables
- Use constants everywhere

- Separate business logic into services
- Keep controllers clean
- Use reusable utilities
- Validate every request
- Never trust frontend

FINAL CONCLUSION

This backend architecture is designed for:

- Realtime multiplayer gameplay
- Scalable socket systems
- Secure wallet handling
- Admin panel integration
- Bot gameplay
- Matchmaking
- Competitive gaming ecosystem

This document covers:

- Complete backend APIs
- Socket events
- Services
- Middleware
- Security
- Validation
- Deployment
- Scalability
- Database design
- Realtime systems
- Admin systems
- Matchmaking systems
- Wallet systems